

Decisiones

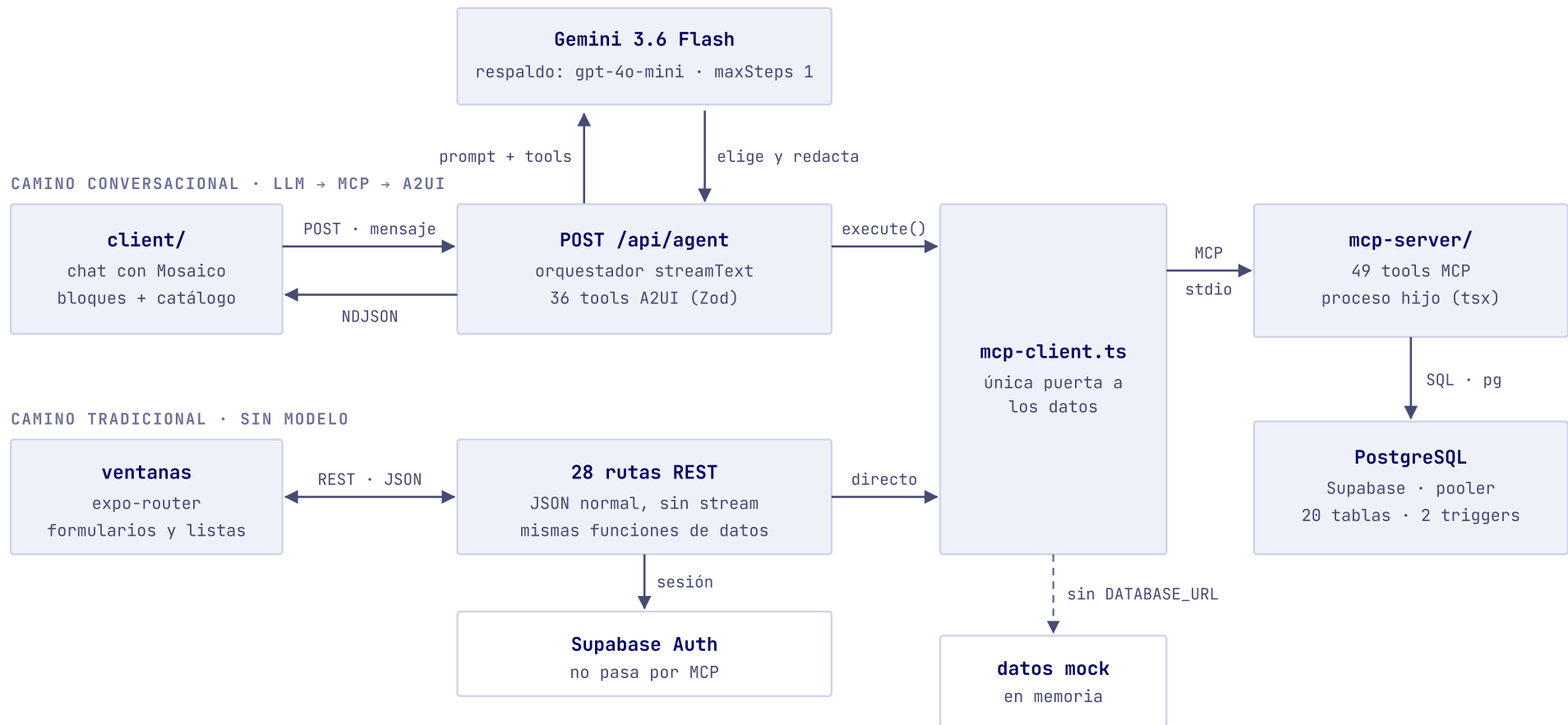
Banortech es la app; **Mosaico** es el agente que vive dentro de ella. Tres paquetes de Node independientes y un contrato que no se rompe entre ellos. Esto es cómo viaja una pregunta del teléfono a Postgres — y qué pagamos por cada decisión del camino.

3 paquetes | 49 tools MCP | 36 tools A2UI | 29 rutas HTTP | 20 tablas | 16 componentes en catálogo | 4 tipos de evento

La arquitectura

CLIENT / SERVER / MCP-SERVER

El asistente no es la app: es un canal más. Un usuario de banco espera poder ver su saldo o hacer una transferencia sin escribirle a un chat, así que el diagrama tiene **dos carriles**, no uno. El conversacional pasa por el modelo; el tradicional es un formulario contra REST. Los dos terminan en la misma puerta – `lib/mcp/mcp-client.ts` – así que la cifra que ves sale del mismo lugar, hayas preguntado o hayas tocado un botón.



Dos entradas, una sola fuente de datos. El modelo **nunca** ve las tools crudas del MCP: solo las 36 tools A2UI, que son las que sí llaman a **mcp-client.ts**. Si falta **DATABASE_URL**, esa misma puerta responde con mocks en memoria y el resto de la app no se entera.

Lo que cruza la red del carril de arriba son cuatro tipos de evento, uno por línea. Nada más. Agregar un quinto obliga a cambiar primero el contrato escrito (`constitution.md` §4.1), no el código.

```
{"type":"text","content":"Vas por buen camino con tu fondo."}
{"type":"surface","tipo":"RastreadorMetas","props":{"titulo":"Fondo de emergencia","porcentaje":62, ... }}
{"type":"done"}
{"type":"error","message":" ... "}
```

Una línea `surface` es un bloque ya resuelto: el servidor dice *qué* componente y con qué datos, el cliente decide *cómo* se ve. El porcentaje de ahí no lo escribió el modelo – lo eligió por referencia (`metaId`) y el código inyectó el número que vino de Postgres.

Los trade-offs

QUÉ ELEGIMOS, QUÉ NOS CUESTA

Modelo EL ORQUESTADOR

ELEGIMOS	Gemini 3.6 Flash como primario y gpt-4o-mini como respaldo, los dos entrando por <code>@ai-sdk/*</code> para que el mismo <code>streamText({ tools })</code> sirva para ambos.
POR QUÉ	El free tier de Gemini da alrededor de 20 requests al día por modelo. En una demo en vivo, un <code>429</code> es el final de la presentación. Como los dos proveedores comparten interfaz, el respaldo conserva el tool-calling: no degrada a un chatbot que solo escribe texto, sigue pudiendo generar bloques. Usamos el id explícito <code>gemini-3.6-flash</code> y no el alias <code>gemini-flash-latest</code>, porque el alias se mueve solo y con él se mueve la cuota.
CUESTA	El respaldo solo aplica si el proveedor falla antes de emitir contenido. A media respuesta no se reintenta: reiniciar duplicaría lo que el usuario ya está leyendo. Y <code>maxSteps: 1</code> no es una optimización, es una restricción heredada: Gemini 3 exige un <code>thought_signature</code> al mandar de vuelta resultados para una segunda ronda de tools, y el <code>@ai-sdk/google</code> instalado no lo propaga – reventaba <i>después</i> de haber pintado las tarjetas, así que el usuario veía su UI y luego un error rojo. Varios bloques por turno salen igual, con llamadas paralelas en una sola ronda.

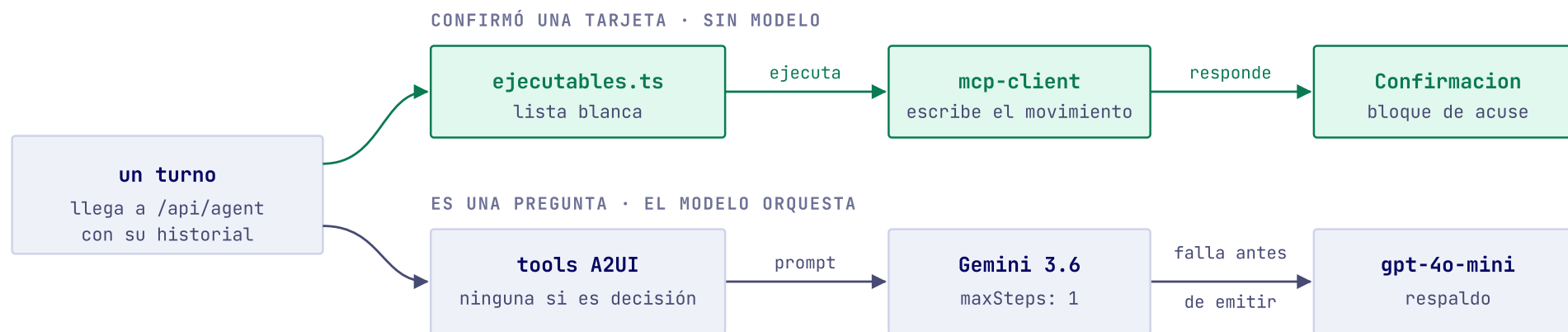
LO QUE EL MODELO NO DECIDE

En el turno de confirmación le quitamos las tools. Medido: sin eso, «acepto el plan» hacía que volviera a invocar `proponerPlanAhorro` en 2 de cada 3 intentos – se parece demasiado a «quiero un plan», y la descripción de la tool le gana al system prompt. Pedírselo por prompt es apostar a que obedezca; quitarle la herramienta lo vuelve imposible.

El dinero no pasa por el modelo. La tarjeta viaja con una receta `{tool, args}` ya validada; al confirmar, el cliente la devuelve y el servidor la ejecuta desde una lista blanca, con el `userId` puesto siempre por el servidor. Volver a preguntarle al modelo abriría la puerta a que se equivoque de contacto justo en el paso que mueve el dinero.

SE REVISA CUANDO

Haya presupuesto para un modelo de pago como primario, o cuando el proveedor propague `thought_signature` y podamos encadenar rondas de tools de verdad (consultar ids y luego referenciarlos).



El carril de arriba es el que mueve dinero, y por eso **no toca el modelo**: el monto y el contacto ya quedaron fijados cuando se propuso la tarjeta. El de abajo es el único donde el LLM decide algo – y decide **qué mostrar**, nunca cuánto.

Protocolo

LO QUE CRUZA LA RED

ELEGIMOS

A2UI-lite: NDJSON, un bloque ya resuelto por línea (`{type, tipo, props}`), interpretado por un catálogo que vive en el cliente.

POR QUÉ

No existe SDK de A2UI para React Native – hay para Web, Angular y Flutter. Y la versión anterior de este repo usaba `streamUI` con React Server Components, que manda **JSX ya renderizado**: un formato que solo React DOM sabe pintar. Se eliminó por completo.

Mandar JSON declarativo es exactamente lo contrario: el servidor nombra el bloque, el cliente decide cómo se ve, y el mismo evento sirve en Android, iOS y web sin mantener dos implementaciones.

CUESTA

Es un subconjunto fiel, no A2UI completo: **no hay streaming granular** componente por componente. Cada bloque aparece entero o no aparece.

Y el nombre del componente es un string acoplado a los dos lados: renombrar `TarjetaSaldo` obliga a tocar el catálogo del cliente y la tool del servidor en el mismo commit, porque nada lo verifica en tiempo de compilación.

A CAMBIO

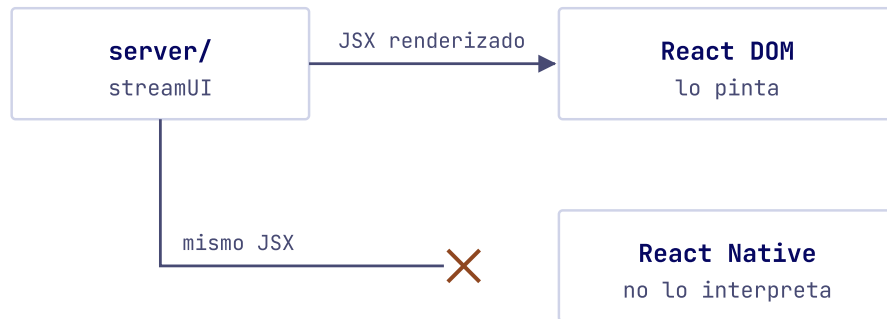
Agregar un bloque nuevo es **una línea** en `catalog.ts` . No se toca el parser del stream, ni el renderer, ni ninguna pantalla. Si para meter un bloque hiciera falta editar una ventana, el contrato ya se rompió.

Y la regla dura del protocolo: el modelo referencia los datos por `idDato` , nunca los transcribe. Una cifra que el modelo escribe como texto –aunque sea «solo para pasarla»– es una cifra que se puede equivocar.

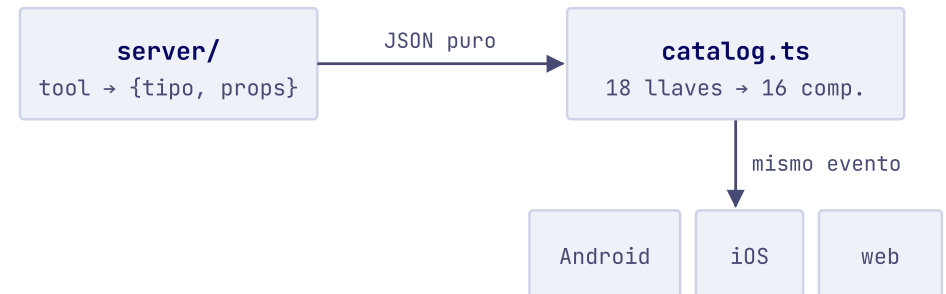
SE REvisa CUANDO

Salga un SDK oficial de A2UI para React Native, o cuando un bloque sea tan pesado que valga la pena mandarlo por partes.

ANTES · streamUI / RSC – RETIRADO



HOY · A2UI-LITE SOBRE NDJSON



La diferencia es una sola arista: lo que viaja. **JSX renderizado** solo lo puede pintar quien tiene React DOM; **JSON declarativo** lo puede pintar cualquier cliente que tenga su propio catálogo.

Infraestructura DÓNDE CORRE Y QUÉ FALTA

ELEGIMOS

Tres paquetes de Node independientes, sin herramienta de monorepo. `mcp-server` se levanta como proceso hijo por **stdio**; Postgres vive en Supabase detrás del session pooler; el historial del chat se queda en el dispositivo.

POR QUÉ

Stdio no necesita desplegar nada ni abrir un puerto: se clona el repo, se corre `node scripts/dev.js` y el handshake MCP ocurre solo. Y sin `DATABASE_URL`, `mcp-client.ts` responde con datos en memoria – el equipo entero pudo construir cliente y agente completos antes de que existiera la base.

Como el historial vive en el teléfono, `/api/agent` es **stateless** por diseño: el cliente manda sus turnos en cada request (tope de 20 turnos y 2000 caracteres por turno, porque un historial sin tope es una factura de tokens abierta).

CUESTA

Stdio no sobrevive a serverless. Un proceso hijo por instancia no es un modelo de despliegue. Al salir de una laptop hay que cambiar el transporte a SSE/HTTP contra un `mcp-server` desplegado aparte: es el pendiente conocido más grande que tenemos.

Además hoy sigue encendida `AUTH_USUARIO_SIN_TOKEN`, la variable temporal que atiende como `demo-user` a quien llegue sin token. Existe para que las pantallas tradicionales muestren datos reales mientras el login no emite tokens – la alternativa era dejarlas con constantes escritas a mano, que se ven bonitas y mienten. Se apaga antes de cualquier despliegue.

FRONTERAS A PROPÓSITO

Auth no pasa por MCP. MCP es la capa de datos financieros, no un gateway de identidad; el login vive donde vive el de cualquier app normal.

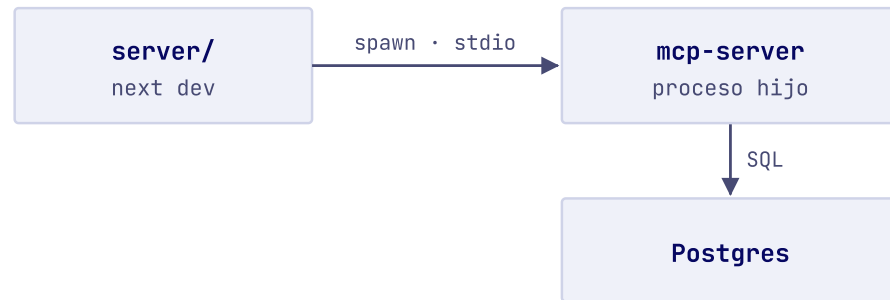
El dashboard guarda la receta, no el número. Cuando el usuario ancla un bloque, se guarda `{componente, tool, parametros}` para volver a ejecutarlo, no el resultado. Un 62% guardado hoy es un dato falso la semana que entra – y sería falso sin que nadie lo haya inventado.

LO QUE YA NOS COSTÓ

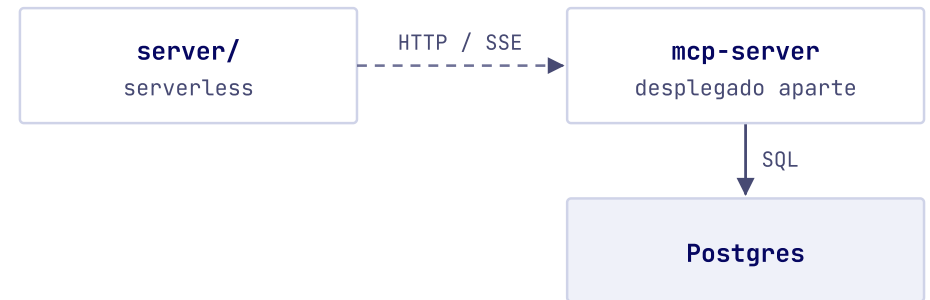
`spawn('npx')` truena en Windows: ahí `npx` es un `.cmd` y `child_process.spawn` sin shell no lo puede ejecutar. Se resolvió invocando `node.exe` directo sobre el archivo real de `tsx`.

En Supabase, la conexión directa resuelve solo por IPv6 y falla con `getaddrinfo ENOENT` según la red; la del pooler sí resuelve por IPv4. El esquema –20 tablas y dos triggers que mantienen el saldo sincronizado con los movimientos– está verificado en vivo contra esa base, no solo en papel.

HOY · UNA LAPTOP



AL DESPLEGAR · PENDIENTE



Lo único que cambia al desplegar es el **transporte**: de **stdio** a **SSE/HTTP**. Las 49 tools, el esquema y el contrato A2UI siguen igual – por eso el cambio está acotado a un archivo, **mcp-client.ts**.

Lo que no se negocia

CONSTITUTION.MD §6

Cuatro reglas escritas antes de que el equipo se dividiera en ramas. Romper cualquiera exige cambiar el documento primero, en un commit aparte, explicando por qué.

-
- 01 Nunca viaja JSX, HTML ni código ejecutable del servidor al cliente — solo JSON.
 - 02 Ninguna cifra financiera la genera el modelo: monto, porcentaje y fecha salen de `mcp-server/`.
 - 03 Ningún componente del cliente importa nada de `server/` ni hace `fetch` : recibe props y se pinta.
 - 04 Ninguna dependencia de UI entra en `server/` : esa capa ya no tiene interfaz propia.
-